*Sohaa Jamadar | 1252090024 | 24 | SEDA-B1 | Assignment 11*

## Import required libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion
```

## Load Dataset

```
df = pd.read_csv('Admission_Predict.csv')
df.head()
```

| | Serial No. | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | Research | Chance of Admit |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 337 | 118 | 4 | 4.5 | 4.5 | 9.65 | 1 | 0.92 |
| **1** | 2 | 324 | 107 | 4 | 4.0 | 4.5 | 8.87 | 1 | 0.76 |
| **2** | 3 | 316 | 104 | 3 | 3.0 | 3.5 | 8.00 | 1 | 0.72 |
| **3** | 4 | 322 | 110 | 3 | 3.5 | 2.5 | 8.67 | 1 | 0.80 |
| **4** | 5 | 314 | 103 | 2 | 2.0 | 3.0 | 8.21 | 0 | 0.65 |

Next steps: Generate code with df New interactive sheet

## Data Preprocessing

```
df.info()
if 'Serial No.' in df.columns:
    df.drop(columns=['Serial No.'], inplace=True)

df['Admitted'] = df['Chance of Admit '] >= 0.75
df['Admitted'] = df['Admitted'].astype(int)
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Serial No.         400 non-null    int64
 1   GRE Score          400 non-null    int64
 2   TOEFL Score        400 non-null    int64
 3   University Rating  400 non-null    int64
 4   SOP                400 non-null    float64
 5   LOR                400 non-null    float64
 6   CGPA               400 non-null    float64
 7   Research           400 non-null    int64
 8   Chance of Admit    400 non-null    float64
dtypes: float64(4), int64(5)
memory usage: 28.3 KB
```

| | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | Research | Chance of Admit | Admitted |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 337 | 118 | 4 | 4.5 | 4.5 | 9.65 | 1 | 0.92 | 1 |
| **1** | 324 | 107 | 4 | 4.0 | 4.5 | 8.87 | 1 | 0.76 | 1 |
| **2** | 316 | 104 | 3 | 3.0 | 3.5 | 8.00 | 1 | 0.72 | 0 |
| **3** | 322 | 110 | 3 | 3.5 | 2.5 | 8.67 | 1 | 0.80 | 1 |

Next steps: Generate code with df New interactive sheet

Missing value to check

```
df.isnull().sum()
```

| | 0 |
|---|---|
| **GRE Score** | 0 |
| **TOEFL Score** | 0 |
| **University Rating** | 0 |
| **SOP** | 0 |
| **LOR** | 0 |
| **CGPA** | 0 |
| **Research** | 0 |
| **Chance of Admit** | 0 |
| **Admitted** | 0 |

**dtype:** int64

## Feature Selection

```
X = df.drop(columns=['Chance of Admit ', 'Admitted'])
y = df['Admitted']
```

## Train Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (320, 7)
Testing size: (80, 7)
```

## Decision tree model

```
model = DecisionTreeClassifier(criterion='gini',splitter="best",min_samples_
model.fit(X_train, y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

## Prediction

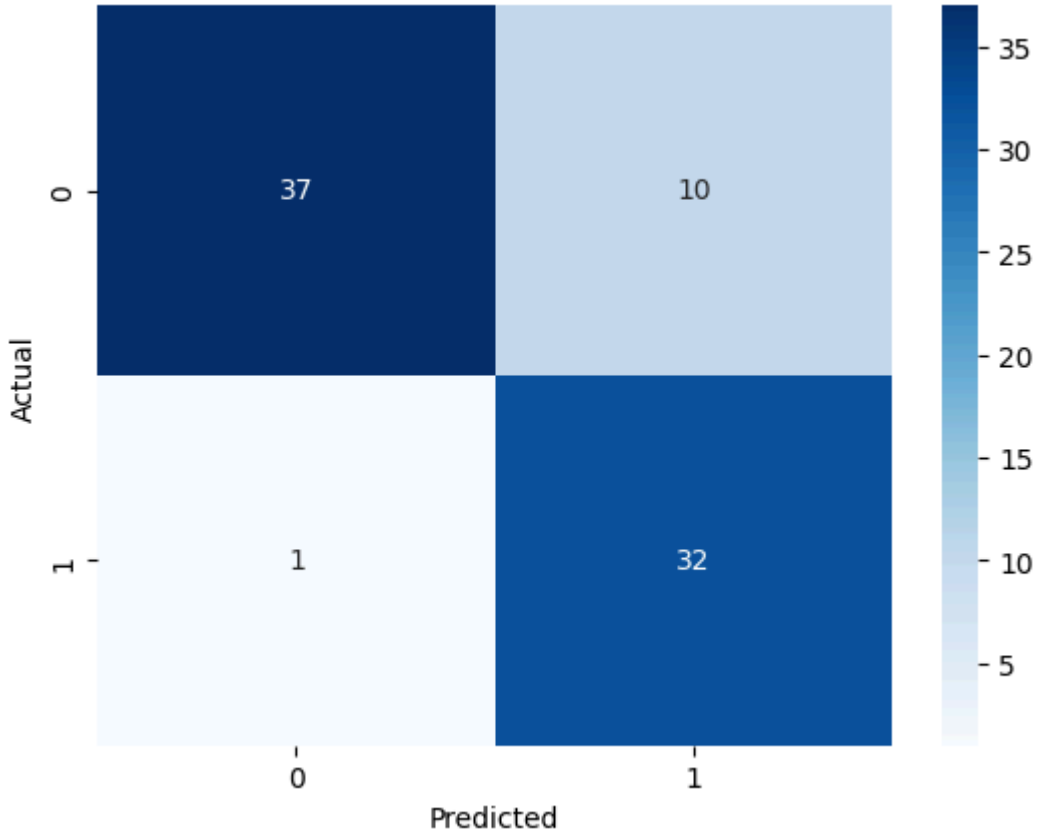
```
y_pred = model.predict(X_test)
y_pred
```

```
array([0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0,
       1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0,
       0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1])
```

## Model Evaluation

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.8625
```



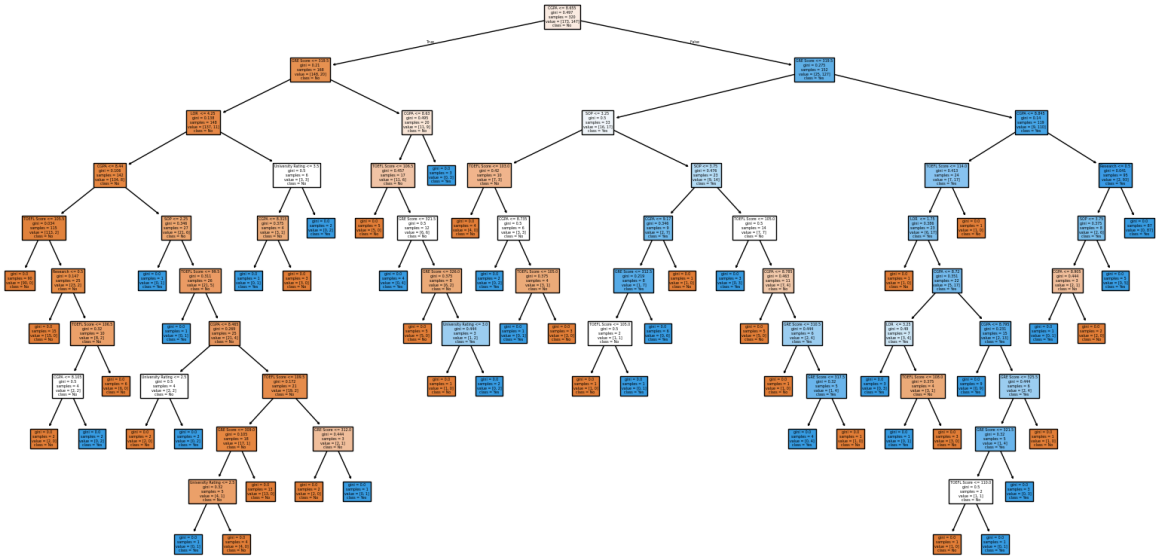

```
              precision    recall  f1-score   support

           0       0.97      0.79      0.87        47
           1       0.76      0.97      0.85        33

    accuracy                           0.86        80
   macro avg       0.87      0.88      0.86        80
weighted avg       0.89      0.86      0.86        80
```

## Visualization

```
from sklearn.tree import plot_tree
plt.figure(figsize=(20,10))
plot_tree(model, feature_names=X.columns, class_names=['No', 'Yes'], filled=
plt.show()
```



## Random Forest classifier

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
```

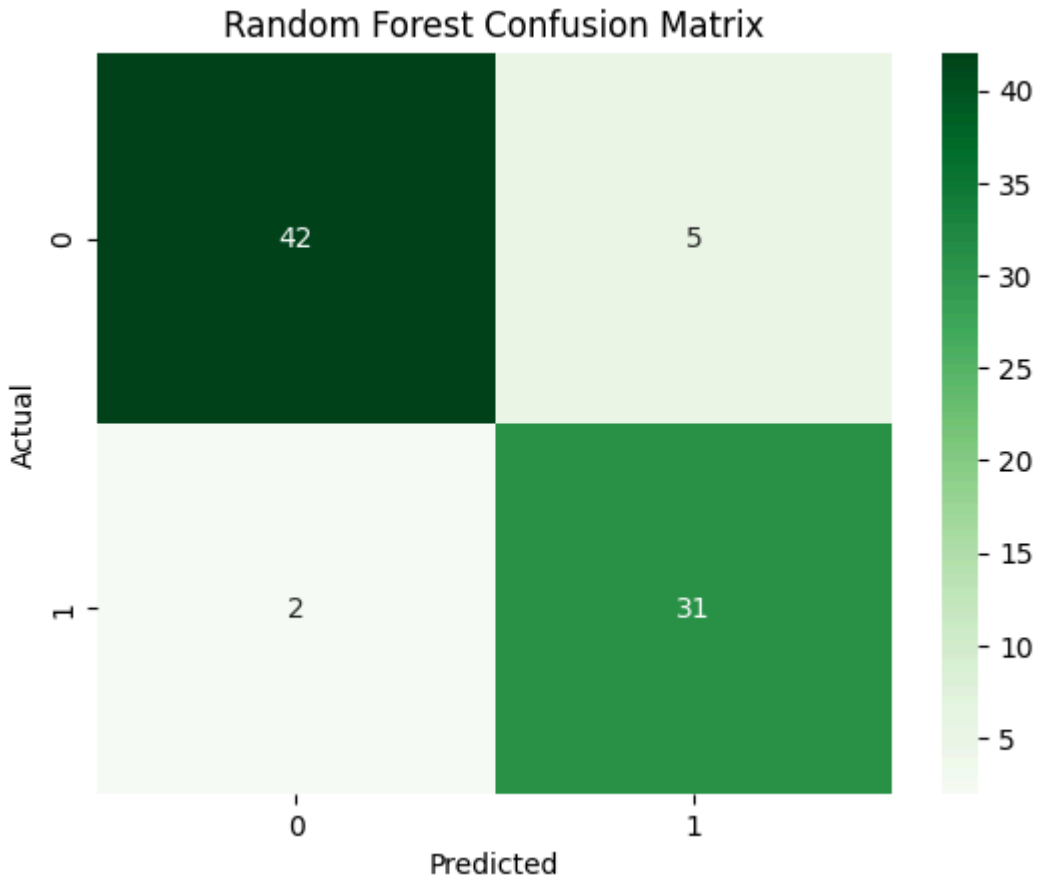
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
rf_y_pred = rf_model.predict(X_test)
rf_y_pred
```

```
array([0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1])
```

```
rf_accuracy = accuracy_score(y_test, rf_y_pred)
print("Random Forest Accuracy:", rf_accuracy)
rf_cm = confusion_matrix(y_test, rf_y_pred)
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Greens')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")
plt.show()
print(classification_report(y_test, rf_y_pred))
```

```
Random Forest Accuracy: 0.9125
```

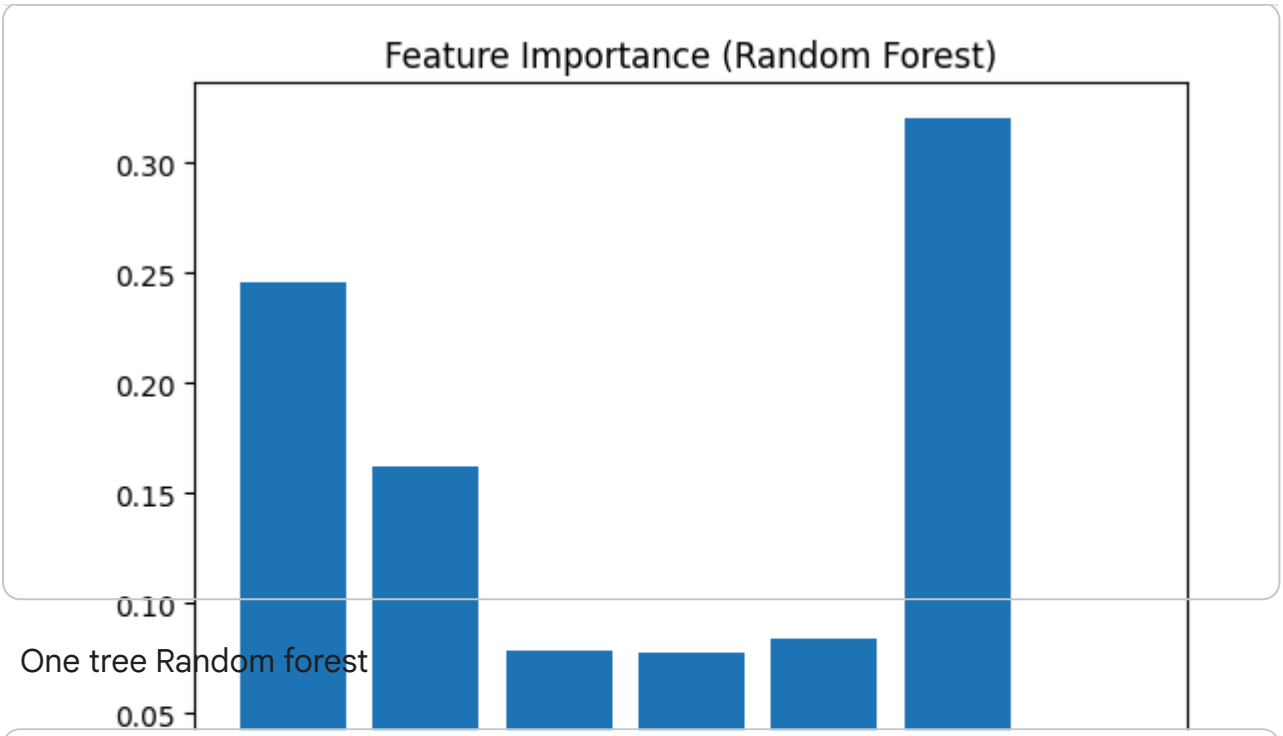



```
              precision    recall  f1-score   support

           0       0.95      0.89      0.92        47
           1       0.86      0.94      0.90        33

    accuracy                           0.91        80
   macro avg       0.91      0.92      0.91        80
weighted avg       0.92      0.91      0.91        80
```

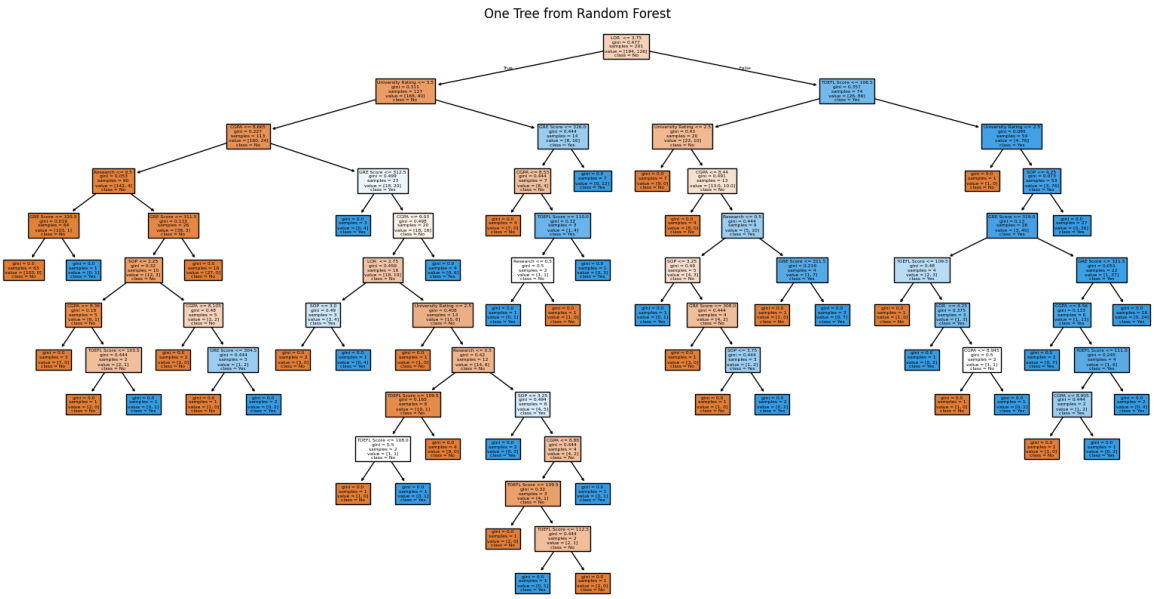
```
importances = rf_model.feature_importances_
features = X.columns
plt.figure()
plt.bar(features, importances)
plt.xticks(rotation=45)
plt.title("Feature Importance (Random Forest)")
plt.show()
```




One tree Random forest

```
from sklearn.tree import plot_tree
one_tree = rf_model.estimators_[0]
plt.figure(figsize=(20,10))
plot_tree(one_tree, feature_names=X.columns, class_names=['No', 'Yes'], fille
plt.title("One Tree from Random Forest")
```

```
Text(0.5, 1.0, 'One Tree from Random Forest')
```




Comparison of accuracies of DecisionTree and RandomForest

```
print("Decision Tree Accuracy:", accuracy)
print("Random Forest Accuracy:", rf_accuracy)
```